

01. 머신러닝의 개념

머싱러닝

- 애플리케이션을 수정하지 않고도 데이터 기반으로 패턴을 학습하고 결과를 예측하는 기법
- 복잡한 조건으로 인해 기존의 소프트웨어 코드만으로 해결하기 어려웠던 문제들을 머싱러닝이 해결
 - 예: 스팸메일 필터링 프로그램을 만들 때 특정 단어가 포함된다고 스팸으로 분류 불가능 → 이러한 문제를 머신러닝은 데이터 기반으로 숨겨진 패턴을 인지해 해결함
- 데이터 분석 영역은 재빠르게 머신러닝 기반의 예측 분석(Predictive Analysis)으로 재편
 - 데이터에 감춰진 새로운 의미와 인사이트를 발굴
- 난이도와 개발 복잡도가 너무 높아질 수 밖에 없는 분야들(데이터마이닝, 영상 인식, 음성 인식, 자연어 처리)에서 머신러닝이 급속하게 발전하고 있음

분류

- 지도학습(Supervised Learning)
 - 분류(Classification)
 - 회귀(Regression)
 - 추천 시스템
 - 시각/음성 감지/인지
 - 텍스트 분석, NLP
- 비지도학습
 - 클러스터링
 - 차원 축소(Dimensionality Reduction)
 - 강화학습(Reinforcement Learning)

데이터 전쟁

- 머싱러닝의 단점 - 데이터에 의존적

- 가비지 인, 가비지 아웃 : 좋은 데이터를 갖추지 못한다면 머신러닝의 수행 결과도 좋을 수 없다
- 머신러닝 모델을 개선하기 위해서 최적의 머신러닝 알고리즘과 모델 파라미터를 구축하는 능력보다 데이터를 이해하고 효율적으로 가공, 처리, 추출해 최적의 데이터를 기반으로 알고리즘을 구동할 수 있도록 준비하는게 더 중요함
- 다양하고 광대한 데이터를 기반으로 만든 머신러닝 모델이 품질 더 좋음

파이썬과 R 기반의 머신러닝 비교

- 파이썬과 R 둘다 대표적인 오픈 소스 프로그램 언어
- 컴파일러 기반의 언어들(C,C++,Java)도 가능하지만 파이썬과 R에 비해
 - 개발 생산성이 떨어짐
 - 지원 패키지와 생태계가 활발하지 않음
 - 그러나, 즉각적인 수행 시간이 중요한 머신러닝 애플리케이션에는 적용이 활발
- R
 - 통계 전용 프로그램 언어
- 파이썬
 - 다양한 영역에서 사용되는 개발 전문 프로그램 언어
 - 유연성 높음
 - 직관적인 문법
 - 객체지향
 - 함수형 프로그래밍
 - 다양한 라이브러리
- 개발 언어에 익숙하지 않으나 통계 분석에 능한 현업 사용자라면 R, 머신러닝을 시작하려는 개발자라면 파이썬 권장
- 파이썬이 R에 비해 뛰어난 점들
 - 쉽고 뛰어난 개발 생산성
 - 오픈 소스 계열의 전폭적인 지원

- 인처프리터 언어의 특성상 속도는 느리지만, 뛰어난 확장성, 유연성, 호환성으로 다양한 영역에 사용되고 있음
- 머신러닝 애플리케이션과 결합한 다양한 애플리케이션 개발 가능
- 다양한 기업 환경으로의 확산 가능
- 딥러닝 프레임워크 텐서플로, 케라스, 파이토치등에서 우선 정책으로 파이썬 지원

02. 파이썬 머신러닝 생태계를 구성하는 주요 패키지

주요 패키지

- 머신러닝 패키지
 - 사이킷런(Scikit-Learn): 데이터 마이닝 기반의 머신러닝에서 독보적
 - 텐서플로, 케라스 : 딥러닝 라이브러리
- 행렬/선형대수/통계 패키지
 - 넘파이(NumPy): 행렬과 선형대수를 다룸, 행렬 기반의 데이터 처리
 - 사이파이(SciPy): 자연과학과 통계를 위한 다양한 패키지
- 데이터 핸들링
 - 판다스: 대표적인 데이터 처리 패키지, 2차원 데이터 처리, 맷플롯립을 호출해 쉽게 시각화 가능
- 시각화
 - 맷플롯립: 세분화된 API로 익히기 어려움
 - 시본: 판다스와의 쉬운 연동, 더 함축적인 API, 분석을 위한 다양한 유형의 그래프/차트 제공
- 서드파티 라이브러리
- 아이파이썬(IPython, Interactive Python)
 - 대화형 파이썬 툴: 전체 프로그램에서 특정 코드 영역별로 개별 수행을 지원
 - 대표적인 아이파이썬 지원 툴: 주피터 노트북

03. 넘파이(NumPy)

넘파이란?

- **Numerical Python**을 의미하는 넘파이(NumPy)는 파이썬에서 선형대수 기반의 프로그램을 쉽게 만들 수 있도록 지원하는 대표적인 패키지입니다.
- 머신러닝의 주요 알고리즘은 선형대수와 통계 등에 기반하며, 넘파이는 이러한 수학적 연산을 효율적으로 처리할 수 있도록 도와줍니다.

특징

- **빠른 배열 연산:** 루프를 사용하지 않고 대량 데이터의 배열 연산을 가능하게 하여 빠른 계산 속도를 보장합니다.
- **C/C++ 호환 API:** C/C++과 같은 저수준 언어 기반의 호환 API를 제공하여 기존 프로그램과 데이터를 주고받거나 API를 호출해 쉽게 통합할 수 있습니다.
- **과학과 공학 분야에서의 중요성:** 대량 데이터 기반의 과학과 공학 프로그램은 빠른 계산 능력이 매우 중요하며, 파이썬 기반의 많은 과학과 공학 패키지가 넘파이에 의존하고 있습니다.

한계

- **파이썬의 성능 제약:** 넘파이는 매우 빠른 배열 연산을 보장하지만, 파이썬 언어 자체가 가지는 수행 성능의 제약이 있습니다. 따라서 수행 성능이 매우 중요한 부분은 C/C++ 기반의 코드로 작성하고 이를 넘파이에서 호출하는 방식으로 통합할 수 있습니다.
- **데이터 핸들링 기능의 한계:** 넘파이는 배열 기반의 연산 뿐만 아니라 다양한 데이터 핸들링 기능을 제공하지만, 편의성과 다양한 API 지원 측면에서 아쉬운 부분이 많습니다. 특히, 2차원 형태의 데이터에 대한 다양한 가공과 변환, 통계 함수 적용 등에서는 판다스(Pandas)와 같은 다른 패키지에 비해 편리성이 떨어집니다.

활용

- **머신러닝 알고리즘:** 많은 머신러닝 알고리즘이 넘파이 기반으로 작성되어 있으며, 입력 데이터와 출력 데이터를 넘파이 배열 타입으로 사용합니다.

- **다른 패키지와의 연계:** 넘파이가 배열을 다루는 기본 방식을 이해하는 것은 판다스와 같은 다른 데이터 핸들링 패키지를 이해하는 데도 많은 도움이 됩니다.

학습의 필요성

- **마스터하기 위한 시간과 경험:** 넘파이는 매우 방대한 기능을 지원하고 있기에 이를 마스터 하기에는 상당한 시간과 코딩 경험이 필요합니다.
- **머신러닝에서의 중요성:** 넘파이를 이해하는 것은 파이썬 기반의 머신러닝에서 매우 중요합니다. 많은 머신러닝 알고리즘이 넘파이 기반으로 작성되어 있으며, 이들 알고리즘의 입력 데이터와 출력 데이터를 넘파이 배열 타입으로 사용하기 때문입니다.

기본 지식과 API

- **기본 지식:** 넘파이의 기본 지식과 자주 사용되는 API에 대해 이해하는 것이 중요합니다.
- **익숙한 독자:** 넘파이에 익숙한 독자라면 이 부분은 건너뛰어도 됩니다.

넘파이의 기반 데이터 타입: `ndarray`

- 다차원 배열(Multi-dimensional array)을 쉽게 생성하고 다양한 연산을 수행할 수 있도록 지원
 - `np.array()` 함수
 - 파이썬의 리스트와 같은 다양한 인자를 입력받아 ndarray 로 변환
 - `ndarray.shape`
 - 배열의 차원과 크기를 튜플 형태로 나타냄
 - `ndarray.ndim`
 - 차원 확인
- `np.array()` 의 입력 인자
 - 주로 파이썬의 리스트 객체 사용
 - 리스트 `[ ]` 는 1차원 배열을, 리스트의 리스트 `[ [ ] ]` 는 2차원 배열을 표현
- 데이터 타입
 - 숫자 값, 문자열 값, 불 값 등 다양한 데이터 타입을 저장 가능
 - 동일한 데이터 타입만 허용

- ndarray의 데이터 타입은 `dtype`로 확인할 수 있음
- `astype()` 메서드를 사용하여 데이터 타입을 변경할 수 있음

ndarray 생성 및 초기화

- `np.arange()`: 연속된 값을 가진 ndarray 생성.

```
sequence_array = np.arange(10) # [0, 1, 2, ..., 9]
```

- `np.zeros()`: 모든 값이 0인 ndarray 생성.

```
zero_array = np.zeros((3, 2), dtype='int32') # 3×2 크기의 0으로 채워진 배열
```

- `np.ones()`: 모든 값이 1인 ndarray 생성.

```
one_array = np.ones((3, 2)) # 3×2 크기의 1로 채워진 배열
```

reshape()

- 배열의 차원과 크기를 변경.

```
array1 = np.arange(10)
array2 = array1.reshape(2, 5) # 2×5 크기로 변환
```

- 사용: 호환 가능한 크기로 자동 변환.

```
array3 = array1.reshape(-1, 5) # 2×5 크기로 변환
```

인덱싱

- 단일 값 추출: `array[index]`.

```
value = array1[2] # 3번째 값 추출
```

- 슬라이싱: `array[start:end]`.

```
slice_array = array1[0:3] # [0, 1, 2]
```

- **팬시 인덱싱**: 리스트로 특정 위치의 값 추출.

```
fancy_array = array1[[1, 3, 5]] # [1, 3, 5]
```

- **불린 인덱싱**: 조건에 맞는 값 추출.

```
bool_array = array1[array1 > 5] # [6, 7, 8, 9]
```

정렬

- **np.sort()**: 정렬된 배열 반환 (원본 유지).

```
sorted_array = np.sort(array1) # 오름차순 정렬
```

- **ndarray.sort()**: 원본 배열을 정렬.

```
array1.sort() # 원본 배열 정렬
```

- **np.argsort()**: 정렬된 배열의 원본 인덱스 반환.

```
indices = np.argsort(array1) # 정렬된 배열의 인덱스
```

선형대수 연산

- **행렬 내적 (np.dot())**: 두 행렬의 곱.

```
dot_product = np.dot(A, B) # 행렬 A와 B의 내적
```

- **전치 행렬 (np.transpose())**: 행과 열을 교환.

```
transpose_mat = np.transpose(A) # 행렬 A의 전치 행렬
```

04. 판다스(Pandas)

판다스란?

- 판다스는 파이썬에서 데이터 처리를 위해 가장 널리 사용되는 라이브러리입니다.
- 2차원 데이터(행과 열로 구성된 데이터)를 효율적으로 처리할 수 있는 다양한 기능을 제공합니다.
- RDBMS의 SQL이나 엑셀과 유사한 편의성을 제공하며, 데이터 핸들링에 매우 유용합니다.

판다스의 핵심 객체

- **DataFrame**: 행과 열로 구성된 2차원 데이터 구조체. 판다스의 핵심 객체로, 여러 개의 Series로 구성됩니다.
- **Series**: 단일 열(Column)로 구성된 1차원 데이터 구조체. DataFrame의 각 열은 Series입니다.
- **Index**: DataFrame과 Series의 행을 고유하게 식별하는 키 값. RDBMS의 Primary Key와 유사합니다.

DataFrame 생성 및 로딩

- **파일 로딩**: `pd.read_csv()` 를 사용해 CSV 파일을 DataFrame으로 로드할 수 있습니다.

```
titanic_df = pd.read_csv('titanic_train.csv')
```

- **리스트, 딕셔너리, 넘파이 배열로 생성**: `pd.DataFrame()` 을 사용해 리스트, 딕셔너리, 넘파이 배열을 DataFrame으로 변환할 수 있습니다.

```
data = {'Name': ['Chulmin', 'Eunkyung'], 'Year': [2011, 2016]}  
df = pd.DataFrame(data)
```

DataFrame 기본 정보 확인

- `head()` : DataFrame의 상위 N개 행을 출력합니다.

```
titanic_df.head(3)
```


- `shape` : DataFrame의 행과 열 크기를 튜플로 반환합니다.

```
titanic_df.shape # (891, 12)
```

- `info()` : DataFrame의 총 데이터 건수, 데이터 타입, Null 값 개수 등을 확인합니다.
- `describe()` : 숫자형 칼럼의 통계 정보(평균, 최솟값, 최댓값 등)를 제공합니다.

데이터 선택 및 필터링

- **단일 칼럼 선택**: `df['칼럼명']` 으로 단일 칼럼을 선택할 수 있습니다.

```
titanic_df['Age']
```

- **여러 칼럼 선택**: `df[['칼럼1', '칼럼2']]` 로 여러 칼럼을 선택할 수 있습니다.
- **불린 인덱싱**: 조건에 맞는 데이터를 필터링합니다.

```
titanic_df[titanic_df['Age'] > 60]
```

- `loc[]` : 명칭 기반 인덱싱. 행과 열의 이름으로 데이터를 선택합니다.

```
titanic_df.loc[0, 'Name'] # 첫 번째 행의 'Name' 칼럼 값
```

- `iloc[]` : 위치 기반 인덱싱. 행과 열의 위치로 데이터를 선택합니다.

```
titanic_df.iloc[0, 1] # 첫 번째 행, 두 번째 열의 값
```

데이터 정렬

- `sort_values()` : 특정 칼럼을 기준으로 데이터를 정렬합니다.

```
titanic_df.sort_values(by='Age', ascending=False)
```

결손 데이터 처리

- `isna()` : 결손 데이터(Null 값)를 확인합니다.

```
titanic_df.isna().sum()
```

- `fillna()` : 결손 데이터를 특정 값으로 대체합니다.

```
titanic_df['Age'].fillna(titanic_df['Age'].mean(), inplace=True)
```

데이터 그룹화 및 집계

- `groupby()` : 특정 칼럼을 기준으로 데이터를 그룹화합니다.

```
titanic_df.groupby('Pclass')['Age'].mean()
```

- `agg()` : 여러 집계 함수를 동시에 적용합니다.

```
titanic_df.groupby('Pclass').agg({'Age': 'mean', 'Fare': 'sum'})
```

데이터 가공

- `apply()` : lambda 함수를 사용해 데이터를 가공합니다.

```
titanic_df['Name_len'] = titanic_df['Name'].apply(lambda x: len(x))
```